

福尔摩斯：双手机免触控语音转 QQ 消息系统

3099404236^{1*}

¹个人工程项目，2026-08-09

本报告记录一套在普通 Android 权限边界内运行的免触控语音消息系统。用户在手机 2 说出“福尔摩斯”，本地 sherpa-onnx 关键词检测器触发华为输入法语音入口；识别文字只进入待确认状态，用户再次说出“确认发送”后，手机 1 上的 AstrBot 才从 outbox 拉取并发送到 QQ 群，成功后回 ACK。系统同时保留云端整句转写备用链路。报告覆盖原生 Sound Trigger 芯片可用性审计、流式 KWS、个人短语分类器、无障碍手势与后台 Activity 桥接、两阶段提交、华为杀后台恢复、跨网连接、延迟测量、误发送事故及脱敏发布边界。脱敏公开目录的 Python 全量测试为 620/620 通过；50 条私有录音的六折样本外准确率为 0.88。训练样本在 Android 上 50/50 回放通过只作为实现一致性检查，不被解释为泛化成绩。

keyword spotting | Android accessibility | voice interface | AstrBot | reliable messaging

https://github.com/3099404236/holmes-voice-qq-bridge

阅读口径 本文把“已经在目标设备上复现的观察”“代码中的机制”“仍需扩大样本验证的推断”分开书写。文中的私网地址、令牌、设备序列号和群号均使用占位符；个人录音与分类器参数不进入公开仓库。

目标、边界与验收标准

最初的愿望很简单：像“小艺小艺”一样说一句自定义口令，再直接口述群消息，不碰屏幕。但它同时跨越了四个受限边界：第三方应用不能任意改写系统唤醒词；输入法通常不提供公开的语音识别服务；Android 不允许普通应用静默开启辅助功能；QQ 发送会落到另一台手机上的机器人会话。工程目标因此被改写为：在不 root、不替换系统分区、不伪

造系统签名的前提下，完成一条可恢复、可确认、可审计的双手机链路。

最终交付分为三个层次：手机 2 的语音前端、手机 2 的本地接收器、手机 1 的 QQ 发送端。验收不以“某次演示成功”为准，而以状态机安全、失败可重试、服务可观测和代码可脱敏发布为准。

为什么不直接调用讯飞或华为输入法 API.

对已安装输入法的检查显示：没有可供第三方调用的标准 RecognitionService 导出，输入法 subtype 也没有公开语音模式；内部 AIDL 接口不是稳定的公共契约。讯飞独立 SDK 需要开发者 AppID，离线能力又有单独授权。可行路径不是“从后台偷调一个内部函数”，而是在真实输入框中让用户授权的 AccessibilityService 发送系统允许的手势。Android 文档明确说明辅助功能服务由系统绑定，并可在获得对应 capability 后派发手势 [1]。

系统总览

角色分工.

一条消息的完整时序.

1. WakeService 以前台麦克风服务连续读取 16 kHz 单声道 PCM。前台服务必须及时进入前台并展示通知，且新 Android 版本对后台启动和麦克风类型有更严格限制 [2], [3]。
2. KWS 解码出“福尔摩斯”。服务先过个人短语校验，再尝试通过无障碍服务打开 ImeVoiceActivity。
3. 承接页获得真实编辑框焦点，无障碍服务长按华为输入法麦克风。文本稳定 1.5 秒后释放；最终文本再稳定 0.7 秒后 POST /voice/text。

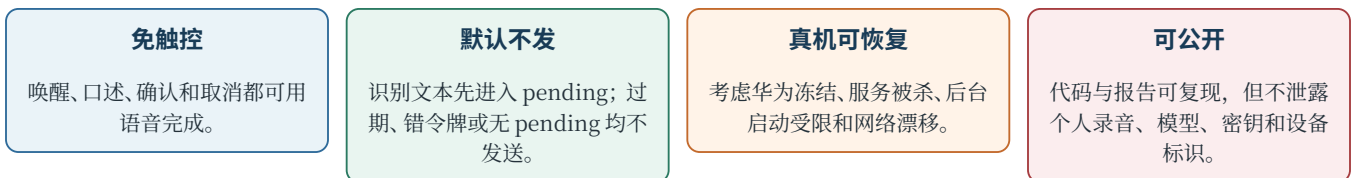
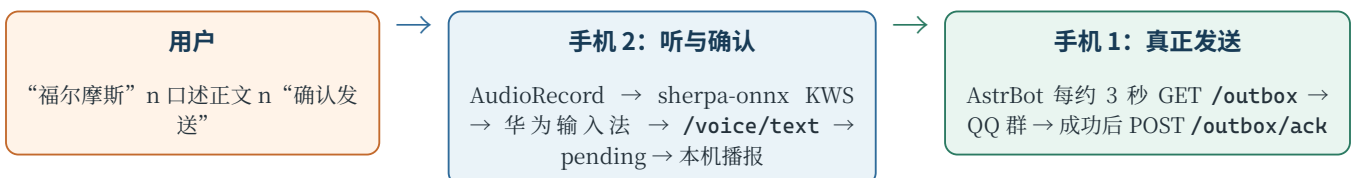


图 1 项目的四项验收标准。



代码 1 从语音到 QQ 的部署拓扑。QQ 会话只存在于手机 1，因此发送端采用拉取而不是由手机 2 直推。

节点	核心进程	职责	关键失败模式
手机 2 / Android	WakeService、无障碍服务、输入法承接页	连续监听三个口令；调起输入法；把文本或 WAV 送到回环地址。	麦克风占用、后台启动受限、辅助功能被清除。
手机 2 / Termux	SpeakServer、speaker_guard.sh	STT/润色备用链路；pending/outbox；TTS 回读；守护后端。	进程被杀、模型/API 超时、环境变量丢失。
手机 1 / proot	AstrBot、imgcard.outbox、插件	拉取已确认条目；调用现存 QQ 会话；成功才 ACK。	地址漂移、代理误路由、QQ 会话未学习。
网络	LAN 或受控的 tailnet	只承载带共享令牌的私有 HTTP。	换 WiFi 后 LAN IP 变化、VPN 分流误判。

表 1 两台手机上的责任分割。

- 服务端拒绝空文本、非法 UTF-8 和过长文本；合法文本进入 pending，生成随机 token，并用不含控制口令的文案读回。
- 用户说“确认发送”。KWS 与个人校验均通过后，本机请求 /voice/confirm。服务端原子取走 pending，放入有界 outbox。
- 手机 1 轮询 /outbox。AstrBot 使用已学习的统一消息来源定位目标群，QQ 发送成功后才 ACK；失败时条目保留，下轮再取。
- 用户说“取消”、token 不匹配、pending 过期或没有待确认内容时，系统不发送。

设计判断 “识别成功”与“发送成功”是两个不同事实。把 pending 和 outbox 分开，才能让识别阶段可撤销、投递阶段可重试。

唤醒词模块：连续流而不是分段云端识别

早期方案让 Termux 每六秒录一段，再把音频送云端找唤醒词。真机很快暴露三个结构性问题：每次启停录音约有 0.5 秒盲区；一句话可能跨分段边界；无关说话也会产生云端请求。termux-microphone-record 输出的 3GP 在结束时才写完索引，无法当作低延迟流消费。最终改为 Android AudioRecord 持续取 PCM，由 sherpa-onnx 流式关键词检测器在本机运行 [4]。

三类口令。

口令	作用	触发后动作	为什么这样选
福尔摩斯	进入输入	打开输入法桥；失败时收 WAV 走备用链路。	四字、语义独特；允许多个常见错听候选。
确认发送	不可逆确认	请求 /voice/confirm。	从单独“发送”扩展为四字，降低日常对话和播报误触发。
取消	安全退出	请求 /voice/cancel 并清空 pending。	短且含义明确；误取消的代价低于误发送。

表 2 当前正式口令。旧的单字/双字发送口令已从关键词表删除。



图 2 候选检测与个人短语分类分层，避免只靠一个宽松 KWS 阈值。

关键词文件不是自然语言列表，而是模型 token 序列。每一条只使用 tokens.txt 中存在的 onset 与带声调韵母；不存在的 token 会导致静默失败。行尾的 boost 越高越容易命中，threshold 越低越容易命中。为了允许用户在口令后立刻接正文，numTrailingBlanks 调为 1，减少必须停顿的要求。系统也保留“小艺小艺”作为已知良好的判别基准，而不是直接替代自定义口令。

双层判定。

个人校验特征为 16 kHz 音频、400 点窗、160 点步长、512 点 FFT、24 个 Mel 滤波器、13 维 MFCC、64 个时间步，最终特征维数 1538。短语分类器不用于鉴权，也不应被称为声纹认证；它只是把两个发音相近且动作代价不同的口令分开。

“原生唤醒芯片”审计

Android 的 Sound Trigger 架构确实允许由 DSP/低功耗硬件处理热词，但可用能力由厂商 HAL、中间件、模型管理和权限共同决定 [5]。存在类名或设置页面，不等于普通应用可以加载自定义模型。

只读探针结果。

探针通过 DexClassLoader 从系统 APK 只读加载类，并把厂商库临时复制到未发布的构建目录；它不改系统设置、不写入系统分区、不注入现有助手。结论不是“所有华为手机都不支持”，而是：在这台机上，普通第三方应用缺少签名级模型权限，HAL 又未暴露所需版本，因此不能靠代码把隐

检查项	实测结果	解释
华为 WakeUpEngine Java 类	可加载并实例化	说明系统 APK 中保留了引擎封装，但不代表完整资源链可用。
自定义“福尔摩斯” enrollment init	-3	训练工厂所需模型/设备能力不满足。
audio	0.0	被测机没有向普通系统路径报告可用的 2.0 能力。
HAL soundtrigger_version		
SoundTriggerManagerEx	对象可创建	读取模型时因 MANAGE_SOUND_TRIGGER 抛出 SecurityException 。
Sound Trigger 模型/事件	0 个模型；8480 行日志中 0 个唤醒事件	隐藏页面没有形成可观察的生产链路。
系统 voice interaction service	指向 Fake service	系统助手入口在该固件上不是可替换的开放服务。

表 3 原生热词能力审计。结果仅描述被测华为设备与固件。

藏开关直接变成自定义“福尔摩斯”。公开仓库只保留原创探针源码，不包含系统 APK、反编译代码和厂商二进制。

语音转文字：借用输入法的真实交互路径

华为输入法语音在两条样句上成功返回文本；最终回调延迟曾测得约 101 ms。日志显示它使用 **CeliaKeyboardOnlineEngine**，因此“输入法在手机上”不等于“整句识别完全离线”。本项目真正离线的是热词检测；正文转写依赖厂商输入法在线能力，备用方案则依赖服务器侧 STT。

为什么要辅助功能权限。

输入法没有提供公共 **RecognitionService**，只能在真实编辑框中长按它的麦克风。普通应用不能向另一个应用窗口任意注入触摸；用户启用的辅助功能服务可以调用 Android 正式的 **dispatchGesture** 接口 [1]。这个权限敏感，所以系统要求用户亲自进入设置开启，应用不能通过普通代码静默打开。辅助功能范围被限制在本流程需要的手势，并且承接页不直接发 QQ，只提交待确认文本。

后台启动 Activity 被拦。

最初即使辅助功能服务已绑定，从后台启动 **ImeVoiceActivity** 仍出现 **START_ABORTED=102**。最终方案先挂一个 2 x 2 像素、不可触摸的 **TYPE_ACCESSIBILITY_OVERLAY**，再由已绑定的辅助功能服务启动承接页；Activity 的 **onCreate** 立即移除桥接 overlay。真机日志显示 window type 2032 获准。它不是可见悬浮窗，也不读取屏幕内容，其唯一作用是为这版固件提供合法的前台交互上下文。

文本稳定判定。

华为输入法会连续更新 partial 文本。承接页采用三段定时：partial 1.5 秒无变化后释放长按；释放后的 final 再稳定 0.7 秒才提交；整次会话 18 秒硬超时。这样既避免第一段 partial 被提前发送，也不会固定傻等十几秒。若辅助功能服务不可用，系统退回本地 WAV 采集并 POST **/voice/audio**，由服务端 **SenseVoice** 转写和 **DeepSeek-V3** 轻度整理。

服务端：从文字到待确认

SpeakServer 监听手机 2 的 8848 端口。Android App 只访问 **127.0.0.1**，手机 1 通过私有网络访问 **outbox**。共享请求令牌放在请求头中；公开版不保存真实值。

接口与信任边界。

Pending 用锁保护 **offer/state/take/drop**。每次 **offer** 生成不可预测 token；新 **pending** 会覆盖旧值并记录 **superseded**；默认 TTL 三分钟；过期会记录 **expired**；**take** 与 **drop** 都拒绝错误 token。确认成功后，**outbox** 项具有独立 ID、文本和时间戳。有界队列防止异常确认无限占内存，溢出会被诊断记录。

STT 与润色的取舍。

备用链路使用云端 **SenseVoice** 转写，再用 **DeepSeek-V3** 只做标点、同音错字和口语清理。曾试过更快的 **Qwen2.5-7B**，但它会删除数字或改变内容；消息发送的第一优先级是忠实，而不是让句子更“漂亮”。本地约 14 MB ASR 在该设备上只节省约 0.4 秒，却明显损害中文准确率，也没有成为默认路径。

接口	方法	作用	安全条件
/voice/text	POST	接收输入法 UTF-8 文本并创建 pending。	共享令牌；长度与编码校验。
/voice/audio	POST	接收 WAV，异步 STT/润色后创建 pending。	共享令牌；WAV 头、空体积与上限校验。
/voice/confirm	POST	原子消费 pending，写入 outbox。	共享令牌或精确 pending token；只消费一次。
/voice/cancel	POST	删除匹配 pending。	token 必须匹配。
/voice	GET	查看当前待确认状态和剩余秒数。	不暴露秘密配置。
/outbox	GET	最多返回一轮上限内的已确认条目。	共享令牌。
/outbox/ack	POST	按 ID 删除已成功投递条目。	共享令牌；重复 ACK 幂等。
/health	GET	守护与部署探活。	不返回敏感值。

表 4 语音消息相关 HTTP 接口。

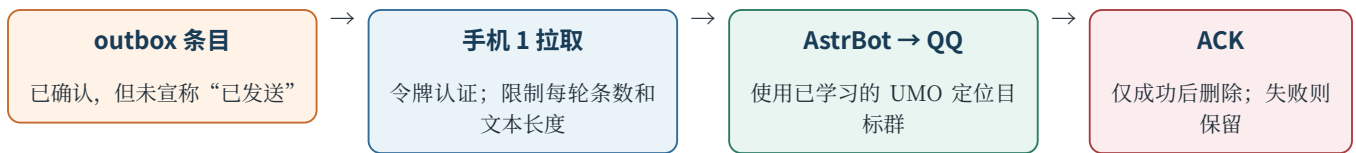


图 3 带 ACK 的 at-least-once 投递路径。插件还用本地已处理 ID 抑制短期重复。



代码 2 误发送事故与三层修复。阈值调整只能缓解, 不能代替结构性隔离。

从 outbox 到 QQ

手机 2 不直接调用 QQ/NapCat。现有 AstrBot 使用由手机 1 维护的反向 WebSocket 会话, 手机 2 没有该会话上下文; 强行复制凭据会扩大权限面且难以确认真实投递结果。因此手机 1 插件每约三秒拉取 outbox。

插件必须先要在目标群看到一条消息, 才能学习 AstrBot 的 unified message origin。若还没学习, 它只记录一次明确警告, 不会 ACK 条目。若 QQ 发送抛错, 诊断中记录 outbox.send, 同一条仍留在手机 2。ACK 失败时, 插件的本地去重集合阻止同进程内重复发送; 跨重启的严格 exactly-once 仍需要持久化幂等键, 这是当前局限。

最重要的事故: 设备听见自己并连续发送

早期确认词是“发送”。系统把识别出的正文读回时, 播报文案自身包含“发送”, KWS 从扬声器再次听到该词, 连续触发三条群消息。这是整条链路中代价最高的故障, 因为它越过了用户确认。

修复包含三项同时成立的约束: 播放活动期间完全暂停检测; 确认播报不得出现任何 App 控制关键词, 且由测试扫描文案; 确认词从常见双字改为四字“确认发送”。线上还做了一个负测试: 只说“发送”, 声音约 -23 dBFS, 没有触发。系统没有选择“好”等常见词, 因为不可逆动作应使用低自然出现率的短语。

华为后台限制与守护恢复

Android 前台服务、常驻通知、wake lock、START_STICKY 和 device-idle 白名单都已使用; 它们符合持续录音服务的基本运行要求 [2], 但不能对抗用户或厂商真正的 force-stop。被测华为固件的 ash 仍会冻结/休眠应用; force-stop 后系统不会仅凭 START_STICKY 自动重建进程, 而且辅助功能启用状态可能被清除。

恢复机制分两层:

- App 内 backend watchdog 定时探测 127.0.0.1:8848/health, 失联时通过 Termux 的

RUN_COMMAND service 请求启动守护脚本。Termux 官方接口要求声明权限、包可见性和允许外部应用调用 [6]。Termux 的 speaker_guard.sh 循环探测服务端, 必要时重启 Python 接收器, 并尝试重启 WakeService。这样 Android 与 Termux 互相守护, 但仍不能绕过系统对真正 force-stop 的安全语义。

部署现实 “常驻”不是一个开关, 而是前台服务、通知、wake lock、电池白名单、厂商启动管理、辅助功能状态和外部 watchdog 的组合。任何一层缺失都可能表现为“好像被杀了”。

网络与代理诊断

双手机链路曾出现手机 1 访问手机 2 的 8848 失败。第一反应是 proot 或 VPN 拦截, 但裸探针证明真正原因是手机 2 换 WiFi 后地址从一个网段漂到另一个网段, 旧 IP 已过期。把同一请求放到 proot 外仍得到完全相同错误, 排除了 proot。

另有两个独立问题: Debian rootfs 的 Python 找不到 CA 路径, 需要设置 SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt 或修复 /usr/lib/ssl; 某外部接口的 Cloudflare 对默认 Python-urllib User-Agent 返回 1010, 改为合理客户端 UA 后恢复。它们与手机间 LAN 故障不能混为一个“网络不通”。

生产环境可使用稳定的私网覆盖地址, 但公开代码只保留占位示例; tailnet 账户、设备清单和真实 100.x 地址不发布。网络层仍需共享请求令牌和 ACL, 不能因为地址“看起来是内网”就视为无认证。

延迟与用户感知

最有价值的优化不是把每个模型快 100 ms, 而是删除固定等待: 采集由固定六秒改成静音自适应; 输入法在 partial 稳定后主动释放; QQ 发送用短轮询而不是等用户再次打开界面。当前最值得继续优化的是确认读回和手机 1 轮询尾延迟。

症状	最初猜测	最小证据	真实修复
8848 Empty reply	VPN/proot 拦截	proot 内外同错；新 IP /health 为 200	更新地址；固定 SSID/DHCP，或使用受控 tailnet。
公网 HTTPS 证书错	网络断开	CA 文件存在但 Python 编译期路径缺失	设置 SSL_CERT_FILE 或补符号链接。
外部 API 403/1010	密钥/模型不可用	仅更换 User-Agent 即 200	显式 UA，并保留状态码诊断。
本机测试统一 502	服务端全坏	环境中 HTTP_PROXY 生效且 NO_PROXY 无 localhost	将 127.0.0.1,localhost 加入 NO_PROXY。

表 5 四类外观相似但根因不同的网络问题。

阶段	实测/配置	解释
离线 KWS	约 25% CPU；实时因子 1.00	持续跟上输入音频，没有积压；代价由一颗 CPU 核的部分时间承担。
结束静音	0.9 s	从固定录满 6 秒改为检测说完，最长可节省约 5 秒。
云端 STT 备用	0.45-4.0 s	取决于音频长度、网络和服务。输入法主路径通常绕过此阶段。
文本润色	1.5-5.4 s	最大波动来自模型服务；可在网络差时降级为原始转写。
确认读回	约 5-8 s	是主观等待最长的一段；应缩短提示、复用缓存或换更快 TTS。
手机 1 轮询	约 0-3 s	均匀情况下平均约 1.5 秒；可用长轮询/推送优化。

表 6 已记录的延迟与配置。它们来自工程日志和真机观察，不是实验室基准。

个人口令训练与评估

采集集包括 30 条“福尔摩斯”和 20 条“确认发送”。有些样本在录音界面刚启动时已提前开口；这些不是错误数据，而是真实使用分布的一部分，只要不是纯静音就保留。训练脚本统一采样率、提取 MFCC、做逻辑回归并导出 Android 可直接读取的 JSON。Android 与 Python 使用相同窗口、Mel 滤波器、DCT 和归一化规则。

三种结果不能混写。

六折混淆矩阵为 $\begin{bmatrix} 28 & 2 \\ 4 & 16 \end{bmatrix}$ ：若行按真实“福尔摩斯/确认发送”、列按预测同序，则唤醒有 2 条被错分为确认，确认有 4 条被错分为唤醒。部署门限故意留下拒绝区：唤醒要求 $P(\text{福尔摩斯}) \geq 0.75$ ，确认要求 $P(\text{福尔摩斯}) \leq 0.70$ 。在不可逆确认上，拒绝并让用户重说比猜错更安全。

现场“确认发送”被 KWS 解码为正确 token 序列，个人概率 0.624，低于 0.70，因此通过确认门。临时诊断录音随后删除；应用设置为 non-debuggable，采集页 `exported=false`。公开仓库只含训练代码和汇总数字，不含 WAV、MFCC 明细或拟合权重。

测试与可观测性

Python 测试覆盖 pending 单次消费、过期、替换、错误 token、取消、outbox 上限、重复 ACK、插件成功后 ACK、

发送失败保留、服务重启后目标群映射、语音 WAV 校验、播放串行化和安全文案。核心语音/outbox 子集为 85/85 通过；补齐公开通用脚本后，脱敏目录的全量套件为 620/620 通过（152.23 秒）。

全量基线第一次得到 528 通过、92 失败，失败请求几乎统一为 502。检查环境发现 HTTP_PROXY/HTTPS_PROXY 指向本机代理，而 NO_PROXY 没有 localhost；测试 HTTP 服务器因此被代理接管。加入 127.0.0.1,localhost 后核心用例全部通过。这一结果没有被写成“代码 92 项失败”，因为证据表明它是测试环境配置错误。公开仓库的 README 把该前置条件写入测试命令。

可观测性采用阶段化事件名，例如 `voice.pending.expired`、`voice.outbox.overflow`、`outbox.fetch`、`outbox.send`。密钥和正文不进入常规诊断快照。手机日志量很大，事后 `logcat -d` 容易错过关键窗口；需要持续采集、心跳和精确 tag。多行 `Log.w` 还曾让文本过滤漏掉第二行，因此安全事件应尽量单行、结构化。

踩坑时间线与可复用经验

这些事故共同说明：移动端系统工程的难点不在单个算法，而在多个“看似成功”的假象。类能反射加载不代表模型能注册；APK 能生成不代表关键类已编译；push 显示成功不代表目标文件存在；`START_STICKY` 不代表 force-stop 后能

评估	结果	能说明什么	不能说明什么
六折样本外	44/50，准确率 0.88	小样本下对未参与该折训练录音的区分能力。	不能代表不同房间、距离、噪声或长期漂移。
Android 训练集回放	30/30 + 20/20；交叉混淆 0	PC 导出与 Android 特征/阈值实现一致。	不是独立测试集，不能宣传为 100% 泛化准确率。
现场诊断	“确认发送”命中；单说“发送”未命中	部署链路和四字负约束至少在两次现场测试中成立。	样本数太少，不能估计误触发率。

表 7 个人短语模型的证据分层。

阶段	现象	根因	最终经验
分段录音	唤醒难、句首丢失	Termux 录音启停盲区与 3GP 末尾索引	热词必须消费连续 PCM；正文录音再做自适应结束。
原生芯片	设置页和类都存在	HAL 版本、模型资源与签名权限缺一不可	先做只读能力探针，不把“藏着”误当成“可用”。
输入法 API	找不到标准识别服务	厂商语音入口是内部实现	使用真实输入框 + 用户授权的无障碍手势。
后台页面	START_ABORTED=102	固件拒绝后台 Activity launch	极小非触摸 accessibility overlay 仅作启动桥。
误发送	设备自听后连发三条	播报词与控制词同域，麦克风回采	播放时停 KWS + 文案禁词 + 四字确认 + 负测试。
APK 构建	安装后没有 WakeService	javac 输出接管道再 true 吞错	构建失败即停，并检查每个关键 .class。

表 8 从唤醒到安全确认的主要失败。

阶段	现象	根因	最终经验
APK 传输	约 40 MB 安装卡住；push 报成功但文件不在	大文件/ADB 状态不稳定	传输后校验大小与 SHA-256，不信单行 success。
配置传输	服务端持续 401	shell 转义把字面 n 拼进口令	base64 传输并两端校验 SHA-256；不回显秘密。
守护脚本	语法检查通过但后半没执行	续行链中插注释截断实际命令	长命令拆函数；验证行为而不仅是 sh -n。
环境变量	重启后配置消失	env -i 白名单漏掉新增变量	新增配置必须同步进入最小环境白名单。
构建环境	JVM/pip 随机失败	系统盘仅剩约 0.14 GB	把磁盘空间纳入构建前检查。
网络	旧 IP 失败被归因 VPN	手机换网，目标地址过期	用同一探针跨边界对照；先验证地址和 /health。

表 9 部署、配置和网络阶段的主要失败。

复活；HTTP 502 不代表业务服务返回 502。每一步都需要独立可观察的完成条件。

复现路径

公开仓库按三块组织：code/android-app、code/python-bridge、code/native-wake-probe。复现时先跑 Python 核心测试，再准备私有模型和 Android 构建环境，最后部署两台手机。

Android 构建脚本从 ANDROID_SDK_ROOT、JAVA_HOME、VOICE_KEYSTORE、VOICE_KEYSTORE_PASSWORD 读取本机信息。LocalConfig.java、个人分类器、ONNX、token 表、JNI/JAR 和签名文件都被 .gitignore 排除。构建脚本会在缺少这些输入时明确失败。

公开发布与隐私边界

公开版本包含原创源码、测试、关键词配置、训练脚本、汇总指标和本报告。不包含：私用项目 ZIP、DPAPI 加密秘密文件、任何真实 API/接收令牌、Android 签名密钥、QQ 群号、手机序列号、局域网或 tailnet 实际地址、个人 WAV、MFCC 报告、个人分类器权重、华为系统 APK、反编译树、厂商库、真机日志与调试截图。

个人短语模型虽不等同于身份认证，但仍从个人声音导出，按隐私敏感资产处理。厂商系统 APK 与库另有版权和许可，不能因为能从自有手机读取就自动获得再分发权。sherpa 模型和运行库也由使用者按上游许可自行下载，仓库只提供项目关键词与集成代码。

步	目标	完成条件
1	服务端测试	NO_PROXY 含 localhost；核心语音/outbox 测试全部通过。
2	个人录音训练	用自己的 WAV 导出 personal_classifier.json；不要提交。
3	模型与 JNI	从 sherpa-onnx 官方来源准备匹配模型、JAR、arm64 库。
4	本地配置	复制 LocalConfig.java.example；两端设置同一随机 receiver token。
5	手机 2	启用麦克风、通知、辅助功能与电池白名单；/health 200；KWS 服务常驻。
6	手机 1	AstrBot 学到目标群 UMO；可拉取空 outbox；发送测试后 ACK。
7	安全验收	播报期间不触发；只说“发送”不触发；取消/过期/错 token 都不发。

表 10 推荐的最小复现顺序。

发布原则 “手机是我的”赋予设备所有者测试和配置权限，但不会把系统签名权限、厂商模型授权或第三方软件再分发权转化为普通应用权限。公开版同时尊重平台安全边界和软件许可边界。

局限与下一步

- **整句识别并非完全离线。**主路径依赖华为输入法在线引擎，备用路径依赖云端 STT/润色。可评估真正可用的本地中文流式 ASR，但必须以准确率、耗电和包体积共同验收。
- **个人数据仍小。**下一轮应按日期、距离、噪声、是否抢话分层采集独立测试集，报告每小时误触发次数与漏唤醒率，而不只报告 clip 分类准确率。
- **确认读回偏慢。**可用短提示音 + 局部文本播报、TTS 预热或更快本地音色，将 5-8 秒等待压缩。
- **轮询有 0-3 秒尾延迟。**可改为长轮询或手机 1 主动保持私有 WebSocket，但仍需 ACK 与重连幂等。
- **跨重启 exactly-once 未完全成立。**outbox 是内存有界队列，插件也主要依赖进程内去重；可引入轻量持久化队列和消息幂等键。
- **辅助功能依赖仍在。**它是当前输入法桥的必要条件。若未来厂商提供公开 RecognitionService，应优先替换手势桥，缩小敏感权限面。
- **华为固件特定。**overlay、坐标与后台策略需要在不同分辨率和固件版本重新标定，不应直接宣称跨品牌通用。

结论

本项目最终没有“破解并改写隐藏唤醒芯片”，而是在普通应用可获得的权限内，把低延迟离线热词、输入法现成转写、严格确认状态机和另一台手机上的 QQ 会话组合起来。最重要的成果不是一次成功演示，而是把不可逆发送从不可靠的语音识别中隔离出来：识别只产生 pending，明确口令才进入 outbox，真实 QQ 成功才 ACK。

工程过程也给出一组可迁移的结论：对私有系统能力先做只读探针；对移动端后台能力接受 force-stop 边界；对语音控制防止自播回采；对跨设备消息保留 ACK；对测试区分代理/环境错误和业务错误；对评估区分训练回放与样本外结果；对公开发布把个人声音特征和厂商二进制留在本机。由此，“福尔摩斯”从一个难唤醒的想法变成了一条可解释、可恢复、可测试且可安全公开的系统链路。

验收域	完成物	可验证条件	状态
语音入口	离线 KWS + 华为输入法桥	“福尔摩斯”可唤醒；输入法文本进入 pending；辅助功能不可用时有 WAV 备用。	完成
发送安全	四字确认状态机	播报时停 KWS；文案无控制词；取消、过期、错 token 和单说“发送”均不发。	完成
跨机投递	outbox + AstrBot + ACK	QQ 成功后才删除；失败保留；目标群未学习时不误 ACK。	完成
工程验证	测试、报告、脱敏仓库	全量 620/620；8 页报告逐页渲染；秘密与个人声音资产不入库。	完成

表 11 公开版本的交付验收摘要。

附录 A：关键配置

变量/文件	位置	作用
IMGCARD_SPEAKER_TOKEN	手机 2 Termux	服务端共享请求令牌。
IMGCARD_PHONE2_HOST	手机 1 AstrBot	手机 2 的稳定私网地址。
IMGCARD_PHONE2_PORT	手机 1/2	默认 8848。
IMGCARD_PHONE2_TOKEN	手机 1 AstrBot	与服务端相同的请求令牌。
LocalConfig.java	手机 2 Android	回环接收器地址与同一令牌；不提交。
personal_classifier.json	Android assets	个人短语分类器；不提交。
keywords.txt	Android assets	KWS 正式短语、候选发音、boost 与 threshold。
speaker_guard.sh	手机 2 Termux	探活并恢复接收器/WakeService。

表 12 部署所需的关键配置；真实值只存在于设备本地。

附录 B：代码索引

文件	阅读入口
WakeService.java	音频循环、播放抑制、KWS 结果、确认/取消、备用 WAV 与后端 watchdog。
VoiceAccessibilityService.java	overlay 桥、后台打开 Activity、长按/释放输入法麦克风。
ImeVoiceActivity.java	真实输入框、文本稳定计时和 /voice/text。
PersonalPhraseVerifier.java	MFCC 与逻辑分类器的 Android 实现。
speaker/voice.py	STT、润色和 Pending。
speaker/server.py	HTTP 路由、TTS、pending、outbox 与 ACK。
imgcard/outbox.py	私网/代理选择、拉取、长度约束和 ACK 客户端。
plugin/main.py	AstrBot 轮询、目标群 UMO、QQ 发送和成功后 ACK。
scripts/train_personal_classifier.py	个人录音特征、交叉验证、阈值与模型导出。
native-wake-probe/ProbeActivity.java	Sound Trigger 和厂商唤醒引擎只读能力探针。

表 13 从报告概念回到源码的索引。

参考资料

[1] Android Developers, 《AccessibilityService API reference》. 见于: 2026 年 8 月 9 日. [在线]. 载于: <https://developer.android.com/reference/android/accessibilityservice/AccessibilityService>

[2] Android Developers, 《Foreground services overview》. 见于: 2026 年 8 月 9 日. [在线]. 载于: <https://developer.android.com/develop/background-work/services/fgs>

[3] Android Developers, 《Restrictions on starting a foreground service from the background》. 见于: 2026 年 8 月 9 日. [在线]. 载于: <https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start>

[4] k2-fsa, 《sherpa-onnx keyword spotting documentation》. 见于: 2026 年 8 月 9 日. [在线]. 载于: <https://k2-fsa.github.io/sherpa-onnx/kws/index.html>

[5] Android Open Source Project, 《Sound Trigger architecture》. 见于: 2026 年 8 月 9 日. [在线]. 载于: <https://source.android.com/docs/core/audio/sound-trigger>

[6] Termux, 《RUN COMMAND Intent》. 见于: 2026 年 8 月 9 日. [在线]. 载于: https://github.com/termux/termux-app/wiki/RUN_COMMAND-Intent